

Detecting Trading Trends in Financial Tick Data

CS-E4780 Scalable Systems and Data Management Course Project

Dennis Magnusson
Aalto University
Espoo, Finland
dennis.magnusson@aalto.fi

Sahel Mesforoosh
Aalto University
Espoo, Finland
sahel.mesforoosh@aalto.fi

Ghazal Shenavar
Aalto University
Espoo, Finland
ghazal.shenavar@aalto.fi

Abstract

In this document we go through our implementation of the course project. We start by having an overview of the problem and then discuss the overall system architecture and our main design choices. In the section following it, the detailed explanation of service implementations are provided alongside the configurations and settings we used to optimize our system. Furthermore, we discuss the performance evaluation and correctness of the system using our static data to ensure we have a reliable system with acceptable latency. Finally, we will discuss the potentials of the project and possible future contributions.

ACM Reference Format:

Dennis Magnusson, Sahel Mesforoosh, and Ghazal Shenavar. 2024. Detecting Trading Trends in Financial Tick Data CS-E4780 Scalable Systems and Data Management Course Project. In . ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Real-time financial tick data can be leveraged by traders to identify trends in markets and support their trading decisions. From the perspective of a trader, it is crucial that the platform used to monitor real-time data is reliable and has low-latency.

In this project, the goal was to design and implement a platform for processing real-time financial tick data and generating alerts for a trader if specific patterns are discovered. The exponential moving average (EMA) of each symbol is used to identify a bullish (buy) or a bearish (sell) breakout pattern to generate alerts.

The scope of the project was focused on the technical aspects of real-time data processing, including data ingestion, processing, and analysis. With limited knowledge of financial trading and limited time constraints, we had to make assumptions about the requirements of the system when designing and implementing our solution.

The DEBS 2022 Challenge dataset [3] was used to generate financial tick events. The dataset consists of 289 million events recorded from Monday, November 8th, to Sunday, November 14th, 2021 on three European exchanges. Each event contains several attributes, but in this project we are mainly concerned about the symbol, security type, last trade price, time of last trade, and date.

The source code for our system is publicly available in a Github repository¹.

¹<https://github.com/dennis-magnusson/TradingAnalyzer>

The sections are divided as follows. In Section 2, we describe our design choices and the motivations behind those choices. In Section 3, we explain how our system was implemented and in Section 4 the evaluation of our system's performance along with correctness is presented. In Section 5 we conclude the report and describe possible future developments to improve the system. In the Contribution section, we outline the contributions for each group member and answer the questions of listed in evaluation document individually in sections with our name.

2 System Architecture

In system design, reliability and performance often exist in a Pareto relationship, where increasing one typically decreases the other [8]. This is especially true in real-time systems, where ensuring absolute reliability—such as guaranteeing that all data is received and processed without duplication—requires additional checks, redundancies, and storage operations that can introduce latency. Conversely, optimizing for performance often entails simplifying processes, tolerating some data loss, and minimizing overhead to achieve faster processing speeds. Recognizing this trade-off is key to designing systems tailored to specific priorities. In our architecture, we chose to prioritize reliability and correctness, as supposed to maximally low latency.

We placed a strong emphasis on accuracy in the query calculations. The exponential moving average (EMA) and breakout pattern detection calculation were validated to ensure that any insights provided to the customer were accurate. This approach ensures that the system meets its primary goal: delivering timely and correct alerts to support effective trading decisions.

We chose to have a modular implementation; both for the possibility of parallelizing implementation and the higher flexibility regarding changes. The modular design of the system can be viewed in Figure 1. The architecture consists of the following key components:

- Data Producer: This module reads the data source and produces the events.
- Data Analyzer: This layer performs the windowing of the incoming trade events and computes the required queries (EMAs and breakout pattern detection)
- Message Broker: An intermediary system ensures reliable communication and coordination between different components, supporting real-time performance.
- Visualizer: Results are presented to the trader through a web interface, with alerts generated for actionable patterns. The user can select which symbols' EMAs they want to view in a line chart.
- Logger: Captures key events during data ingestion and processing, such as timestamped EMA calculations and detected

patterns. This module provides valuable debugging information during system development and testing phases.

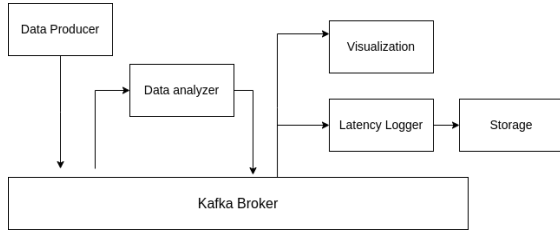


Figure 1: Overview of the system architecture.

3 Implementation

Our system architecture is built around small, independent services that communicate with each other via Kafka Cluster on different topics. This modular design allows each service to be scaled individually based on its specific performance and resource requirements. By decoupling components and leveraging Kafka’s robust messaging capabilities, we achieve both flexibility and efficiency, ensuring the system remains responsive under varying workloads and that we can scale each component according to its specific workload and needs.

In Figure 2 you can see an overview of the system architecture. In the following sections we will go through the services and their responsibilities.

3.1 Containerization

All of the components of our system have been containerized with docker and we have written two different docker compose files for running the system:

- (1) docker-compose.yaml, is meant for running the system on a single machine. It only runs one instance of each component and configures the Kafka topics with a single partition.
- (2) docker-compose-multi.yaml, is meant for running the system with more than one instance for some components. For example, it defines three Kafka controller instances and three Kafka broker instances. In addition, the Kafka topics used, are configured to have up to five partitions and a replication factor of two. Please refer to the source code for exact configurations.

3.2 Kafka

We decided to use Apache Kafka [5] as our distributed event-streaming platform. To insure it’s performance we considered different configurable properties of the Kafka platform.

3.2.1 Partition keys. It is important to note that partitioning highly effects the performance of the Kafka cluster when dealing with multiple instances of a Kafka Streaming application (See Section 3.4 for a description of how we use Kafka Streaming for the stream processing). An uneven distribution of events between partitions will lead to an uneven distribution of work for the Streaming applications and thus non-optimal use of resources.

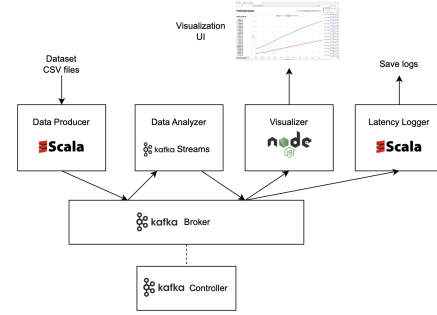


Figure 2: Overview of the system implementation.

In our case, since we need to access each trade event per symbol in each window, we need to make sure that all events regarding a particular symbol are produced by Kafka into the same partition within the Kafka topic. We made the decision to use symbols as partition keys for the input topic, which effectively places all events regarding the same symbol into the same partition within a topic.

We researched the distribution of symbols in one of the provided CSV files to understand how it might affect the performance of our system when using the symbol as the partitioning key.

Looking at the distribution of symbol occurrences for a single day in the dataset (Figure 3), we see that the number of occurrences of different symbols are actually quite uneven. There is a small minority of symbols that occur most often in the dataset. A large number of symbols occur only a handful of times.

Using the symbol as the partitioning key most likely will not lead to the best performance when scaling the system with multiple instances for the Kafka Streaming application. Therefore, the symbol should only be set as the partitioning key when the data analyzer component turns into the performance bottleneck for the entire system.

Possible solutions for this problem are using a specific hashing function in the Kafka Cluster to make sure that each of the high frequency symbols is mapped to a unique partition. Another possible solution would be to add more information to the input data that can be used for filtering and partitioning the events more evenly.

For the scalable architecture with replicas of system components, we limit our number of partitions and brokers to avoid cold partitions and only use symbols as partition key. Adding more information to the input data and working on the hash function of Kafka was something that we decided not to focus on but it is a further development that would enhance the performance of the system.

3.3 Data Producer

The implemented data producer service is a Scala program that reads the provided dataset’s CSV files and produces those trade events into a topic in the Kafka cluster. The implementation ignores any events in the CSV file that contain null values.

For testing purposes, we implemented a way to dynamically change the rate of the event production to the Kafka topic. Data producer serializes the trade events into a comma separated string containing only the symbol, security type, last trade value, trade

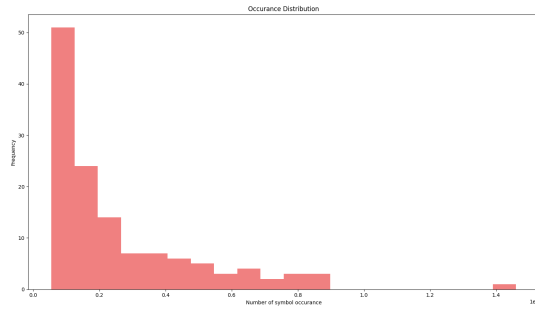


Figure 3: Number of symbols with a specific occurrence in debs2022-gc-trading-day-08-11-21.csv. Occurance on x-axis and number of symbols with that occurrence on the y-axis.

time, and trade date. For the trade time of each event, we ignore the timestamp from the original data and use a timestamp generated at the time of producing the event. Our implementation does, however, allow for timing the intervals between producing events as they are in the original data. Using speed factor environment the user can change the ratio of the actual time interval and the time interval between the produced events.

3.4 Data Analyzer

The data analyzer service does the processing of the data stream.

3.4.1 Technology choice. Our selected choices for this part of the project were Apache Flink, Apache Spark and Kafka Streams. After researching the available tools, we concluded that Apache Spark and Kafka Streams are more reliable, because they are more developed and have better documentation.

We decided to use Kafka Streams, since it is highly supported and integrates natively with Kafka. With Kafka Streams we can also scale our system horizontally relatively easily by running multiple replicas of the application. Kafka Stream takes care of splitting work between replicas based on the partitions within the Kafka topic that the Stream application consumes data from. Therefore, to leverage the benefits of this horizontal scaling, it is crucial to partition the incoming data effectively in the Kafka cluster.

3.4.2 Programming detail. The Kafka Streams application consumes incoming trade events from the input topic for the events. It parses them into TradeEvent class instances from the string format that they are encoded in after deserialization. Those TradeEvent instances are then reduced to only the most recent event for each symbol. Then a Kafka Streams Transformer is used for computing the EMAs for that window and storing the them into a state store for use in computing the next window's EMAs. Once the EMAs have been computed and stored a they are written to a Kafka topic. A Kafka Streams Mapper is used to map the computed EMAs to potential breakout patterns that might be detected. Lastly the potential found patterns are sent to a Kafka topic.

With this implementation, a customer can subscribe to the respective topics in Kafka to receive EMAs and buy and sell alerts directly. By separating the results into two different Kafka topics,



Figure 4: The visualization application running in a web browser from port 8888.

its possible to configure the partitioning separately for each topic according to a customer's needs.

Since we made the choice to value correctness of the EMAs and breakout patterns, we use Kafka Streaming's windowing that relies on event-time rather than processing time. This means that the latency of the EMA calculation and breakout pattern alerting is not the lowest possible, but it will consider events that arrive late or out-of-order using a grace period of 1 second.

3.5 Visualization

The visualization component of the project leverages a web server built using Express.js [2] and Node.js [6]. This server consumes EMA results and buy/sell signals from two Kafka topics via the KafkaJS [4] client library.

A frontend application is served at the web server's root (it can be accessed on localhost:8888) and a web socket connection is established between the server and the browser. As the server consumes events from the two Kafka topics, it transmits them to the frontend, which then utilizes chart.js [1] to dynamically render a line graph for each user-selected symbol. Incoming buy and sell signals are logged to the browser console.

The frontend application allows the user to select which symbols they are interested in and hides other symbols' EMAs. Due to using a WebSocket connection no reloads of the page are required.

A screen capture of the visualization is shown in Figure 4.

4 Performance Evaluation

All the test were ran locally on a laptop with Intel® Core™ i5-8250U CPU @ 1.60GHz × 8 processor and 15.5 GiB of RAM, using Ubuntu 20.04.6 LTS operating system.

4.1 Scalability

To test the system with a higher workload, we introduced a speed factor that affects the rate at which data were provided to the system. Our data generator module simulates the flow of data in the CSV file by sending the records with respect to the intervals between their timestamps. If the speed factor is set to 1, the data generator module replicates the behavior of the tick data. However, by increasing the speed factor, the intervals will be divided by the

aforementioned factor, resulting in a higher volume of data in each window.

To automate testing we implemented the "speed_tester.sh". This script tests the system for speed factors of 100 to 800, with a step value of 100 and reports two latencies. First reported latency is the "EMA Calculation Latency". This latency is the time difference between data analyzer module receiving the data and outputting the EMA value. The second reported latency is "Arrival Latency". This column reports the time difference between the analyzer module sending the data and our alert system receiving it. Table 1 reports the average results of our tests for different values.

Speed Factor	EMA Calculation Latency	Arrival Latency
100	848.00	4.89
200	926.33	5.44
300	654.20	3.70
400	693.70	3.60
500	716.11	3.89
600	616.78	3.89
700	541.40	4.30
800	691.78	3.22

Table 1: The latency for each respective speed factor. The latencies are measured in milliseconds.

The data show that even with a considerable increase in the volume of the input, the latency stays similar.

4.2 Correctness

To test the correctness, we wrote a dummy data generator using Python programming language. The dummy data produced by this script was then used by our batch processing script to generate EMA values and buy/sell alerts. A similar script is available in our data generator module which can be activated by setting the "TEST_STATE" environment variable to true.

Using the script, dummy data was generated for four hours and analyzed. Then, we ran our system for a similar duration. The timestamp of the alerts was compared relative to their starting time and no errors were found. This test was run with the window sizes set to one minute.

The graph corresponding to EMA values for test data is available in Figure 5. The reason this dummy data was used, as it can be seen in the figure, is that after a warming up period the data produces both buy and sell alerts consistently.

4.3 Resource Utilization

The resource utilization of the multi version (docker-compose-multi.yaml) of our code can be viewed in Table 2.

Overtime the memory usage is increasing for latencylogger, controllers and brokers. The memory usage of visualizer is also following an upward curve; however it is around 10kB in 15 minutes.

Brokers have the highest percentage of CPU utilization, followed by data_analyzer and data_producer. Our visualizer and latencylogger use less than 1% of CPU, suggesting a low computational overhead for the system. It is also worth mentioning that despite the

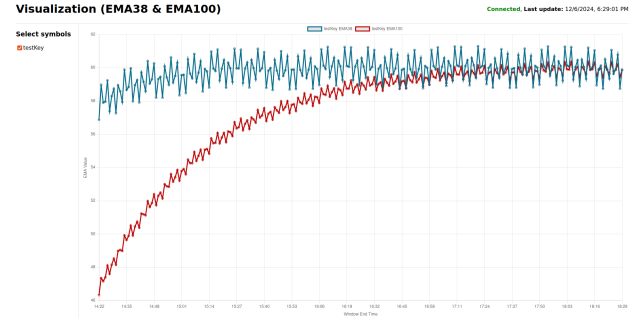


Figure 5: Graph of EMA values for test data.

Container Name	Average CPU Usage	Peak Memory Usage	Network I/O	Block I/O
visualizer	0.00%	18.32MiB	24.9kB	0B
latencylogger	0.25%	224.1MiB	10MB	1.3MB
data_analyzer	36.33%	321.8MiB	427MB	5.5MB
data_producer	25.9%	1.047GiB	144MB	236MB
broker-2	154.99%	1.083GiB	538MB	11.6MB
broker-1	136.88%	1021MiB	493MB	13.3MB
broker-3	121.81%	1.018GiB	638MB	9.34MB
controller-3	1.24%	179.4MiB	1.66MB	705kB
controller-1	4.95%	401.7MiB	5.31MB	934kB
controller-2	0.90%	203.4MiB	2.07MB	827kB

Table 2: Resource utilization metrics reported per container after 30 minutes of execution.

high variance in the CPU usage of brokers, their average converges to a similar point over time.

5 Conclusion

In conclusion, our system's modular architecture, enables seamless scalability and adaptability to evolving workloads. By leveraging reliable and well-documented technologies, we ensured ease of deployment and maintainability across diverse environments. Our performance evaluations show that the system outputs reliable and correct calculations in a timely manner and performs with acceptable latency, even with increasing the number of data produced in each second.

5.1 Future Work

With the limited resources during this project we were able to implement the project on a narrow scope. Therefore we tried to use an architecture that is flexible to update and scale in future based on more specific needs. In this section we will go through the path we have in mind for the future of the project having more resources and time.

Since this implementation is based on a horizontally scalable system, having enough resources we can optimize the number of nodes and partitions depending on the resources. Moreover, we can check the latency and performance information based on different

amounts of resources and number of nodes. In a real world application we will also focus more on the impact of network latency on the system; but, since in the scope of this project we ran the services using docker containers in the same network, we couldn't utilize the network latency parameters of Kafka. Furthermore, this configuration will benefit from adding a monitoring system such as Prometheus [7] to constantly check the system health and resource utilization.

Another important note is the partitioning of event data. As we discussed earlier we need to find an optimal hashing function for getting a partition key for the trade events. This is highly dependent on the number of nodes we decide to have for our Kafka brokers and data analyzer.

We were also planning on creating a Lambda architecture by deploying another system that reads the live data and logs them in InfluxDB for future queries and also connect it to Grafana. However, since our current implementation was sufficient for the main objectives, and we were under time constraints, we left this part for the future. We have the main configurations for reading from the kafka stream and writing to InfluxDB in our project already ready to go.

Finally, in future we plan to add a Schema registry to our system to register our event schema and use the schema from the registry instead of manually agreeing on a specific schema between the producer and analyzer.

6 Contribution

This project was a group work of Dennis Magnusson, Sahel Mesforoosh and Ghazal Shenavar. In order to have smooth process during our work we decided to have weekly sessions in which we discussed previous tasks, analyzed our system architecture and assigned future tasks to each member. Each member of the team contributed to researching system architectures and discussed our conclusions in our meetings.

In the implementation phase, we tested different architectures before finalizing on a specific architecture. Note, that we worked on other implementations before the final version, which included work by team members that are not visible in the final implementation.

6.1 Teamwork and individual contributions

In this subsection, each team member has individually elaborated on the requested questions in the project evaluation table handout.

6.2 Dennis Magnusson

- (1) The most important decisions my team and I made about the system design was whether to approach the project as a low level implementation (written for example in C) to maximize performance and resource use on a single chip or to approach it as a system that is more easily scalable beyond one machine. In addition to making this choice, we decided to use Kafka Streaming instead of Spark Streaming, since Spark Streaming is not a true streaming platform but rather an adaptation of Spark with micro batches.
- (2) The most challenging part of the design phase was to select a starting point. None of us in our group had previous

experience designing a system like this so we didn't know where to start. It was also challenging to get familiar with the architecture of the technologies that we decided to use for implementation. It was also quite difficult to make decisions about windowing, grace periods and out-of-order processing.

- (3) During the system design phase, I researched topics relating to the design choices we made. In the very beginning, I helped our team create a schedule outline for working, and setting incremental tasks. During the implementation phase, I implemented the Kafka Streaming applications stream topology and wrote the queries for EMA calculation. My contributions also included implemented the visualization portion for our system. Our team's overall functionality was good in my opinion. We had regular sessions for planning and working together. I felt that communicating and working with my team was easy.

6.3 Sahel Mesforoosh

- (1) The main important decision was to choose a streaming system with open source technology instead of using low level programming applications or using super computers. The reason we decided to use this design choice was that we believed it is a design that can easily be used for real world application and it is more flexible to configure and optimize for different needs and problems. A lot of streaming applications have the same similar design concepts behind them and learning and using one of them can easily help you to use the other technologies in similar cases which we believed is important for our future and also for the project's goals.
- (2) The most challenging part was to learn the architecture behind these streaming concepts and trying to use them with real world data to calculate complex queries. Since it was our first time working on a streaming application and open source applications sometime lack proper documentation and examples it was hard at first to decide which tools we should use and learn the base structure correctly.
- (3) I believe that our team was highly functional in overall view because despite the challenges and difficulties we occurred and different time tables we managed to split the tasks and communicate with each other effectively. We also had pair programming sessions in which we gathered together to work on the project together so we can help each other and update each other faster. We had 7-8 of these pair programming sessions.

My main contribution was working on the Apache spark at the first step and trying to utilize that. After our decision to change the technology I was mainly focused on learning the important core concepts regarding Kafka and Kafka streams and trying to optimize the parameters based on the architecture and our main goals.

6.4 Ghazal Shenavar

- (1) The most important decision was to prioritize scalability and making the application portable instead of optimizing it for a

specific architecture. This way, all of us could run the system locally on different operating systems and specifications. Moreover, we decided to prioritize automation both for tests and system.

- (2) One of the challenges was which implementation tool to choose first. Depending on which technology we planned to fix first, our choices for other tools would change. Another challenge was finding relevant and reliable resources for debugging the problems, specially during the period that we were trying to tune Spark. We were met with a lack of constructive errors to fix our design.
- (3) My contribution prior to changing our design was researching TimeSeries databases, configuring and connecting InfluxDB to Spark and Kafka, and optimizing writes to InfluxDB. After the change in our tools, I focused on testing the system and writing the documentation. The team was highly functional, we had weekly meetings from the first week the project was announced and multiple peer-coding sessions. During these sessions, we were able to consult other

members of the team without delay and that is one of the main reasons we succeeded in changing our architecture.

References

- [1] Chart.js Contributors. 2024. Simple yet flexible JavaScript charting library for the modern web. <https://www.chartjs.org> accessed: 2024-12-06.
- [2] Express.js Contributors. 2024. Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com> accessed: 2024-12-06.
- [3] Sebastian Frischbier, Jawad Tahir, Christoph Doblander, Arne Hormann, Ruben Mayer, and Hans-Arno Jacobsen. 2022. *DEBS 2022 Grand Challenge Data Set: Trading Data*. <https://doi.org/10.5281/zenodo.6382482>
- [4] KafkaJS Contributors. 2024. KafkaJS - A modern Apache Kafka client for Node.js. <https://kafka.js.org> accessed: 2024-12-06.
- [5] J. Kreps, N. Narkhede, and J. Rao. 2011. Kafka: A Distributed Messaging System for Log Processing. In *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*. Athens, Greece.
- [6] Node.js Contributors. 2024. Node.js® - A free, open-source, cross-platform JavaScript runtime environment. <https://nodejs.org> accessed: 2024-12-06.
- [7] Prometheus Contributors. 2024. Power your metrics and alerting with the leading open-source monitoring solution. <https://prometheus.io/> accessed: 2024-12-06.
- [8] Yu Zang, Jiaxiang E, and Lance Fiondella. 2024. A Network Reliability Analysis Method for Complex Real-Time Systems: Case Studies in Railway and Maritime Systems. *Mathematics* 12, 19 (2024), 3014. <https://doi.org/10.3390/math12193014> This article belongs to the Special Issue on Network Reliability and Optimization of Industrial Engineering and Management.